

CSC 1202 – Lecture 2

Requirements Engineering Project: Internship Logging & Evaluation System (ILES)

Dr Peter Khisa Wakholi (Ph.D)

Why Requirements

- If requirements are weak:
 - Database design will be wrong
 - Workflow logic will break
 - Score computation will fail
 - Dashboards will be inconsistent
- By the end of today, students must:
 - Clearly define user stories for ALL roles
 - Identify workflow states
 - Define system entities
 - Start implementing models

What We Are Building

ILES manages:

- Internship placements
- Weekly logs
- Supervisor reviews
- Academic evaluations
- Weighted score computation
- Dashboards

This is not just a logbook. It is a workflow-driven evaluation system.

Core Roles (From Outline)

You must define requirements for:

- Student Intern
- Workplace Supervisor
- Academic Supervisor
- Internship Administrator

Do not ignore any role.

Writing User Stories

- Remind format:
- As a [role],
I want to [action],
So that [benefit].
- Student Intern Examples: As a student, I want to submit weekly internship logs so that my work can be reviewed.
- Workplace Supervisor: As a workplace supervisor, I want to approve or reject weekly logs so that only valid work is recorded.
- Academic Supervisor: As an academic supervisor, I want to evaluate students using weighted criteria so that grading is standardized.
- Internship Administrator: As an administrator, I want to assign supervisors to students so that each student is monitored.

In-Class Task



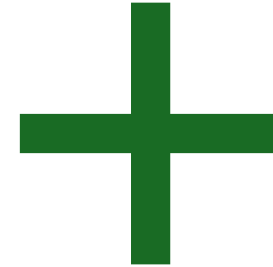
Each team must submit:

5 student stories

3 workplace supervisor stories

3 academic supervisor stories

3 administrator stories



Minimum 14 total.

Functional Requirements

- User story = human intention
- Example: User Story:
"As a student, I want to submit a weekly log."
- Functional requirement = system obligation
- Functional Requirements:
 - The system shall allow a student to create a weekly log.
 - The system shall allow editing while status = Draft.
 - The system shall prevent editing when status = Approved.
 - The system shall timestamp submission.
 - The system shall validate submission deadlines.

In class Task 2:



Convert at least 3 user stories into formal system requirements.



Workplace Supervisor: As a workplace supervisor, I want to approve or reject weekly logs so that only valid work is recorded.



Academic Supervisor: As an academic supervisor, I want to evaluate students using weighted criteria so that grading is standardized.



Internship Administrator: As an administrator, I want to assign supervisors to students so that each student is monitored.

Non-Functional Requirements

Examples aligned to ILES:

- Security: Only assigned supervisors can review a student's logs.
- Integrity: A student cannot have overlapping internship placements.
- Performance: Dashboard loads within 3 seconds.
- Auditability: All review actions must be logged.
- Scalability: System must support at least 500 concurrent users.
- Reliability: No data loss on submission.



Identifying Workflow States

- Weekly Log States:
 - Draft
 - Submitted
 - Reviewed
 - Approved
 - Rejected

- Group Exercise: Try to come up with possible states for evaluation process

Define Core Entities

- You must implement:
 - CustomUser
 - InternshipPlacement
 - WeeklyLog
 - EvaluationCriteria
 - Evaluation

Model Design

- Custom User
 - role (Student, Workplace Supervisor, Academic Supervisor, Admin)
 - department
 - staff_number (optional)
 - student_number (optional)
- Group Exercise 4: Suggest the model for Internship Placement

Creating Models

- CustomUser
 - role (Student, Workplace Supervisor, Academic Supervisor, Admin)
 - department
 - staff_number (optional)
 - student_number (optional)

```
class CustomUser(AbstractUser):  
  
    ROLE_CHOICES = (  
        ('student', 'Student'),  
        ('workplace', 'Workplace Supervisor'),  
        ('academic', 'Academic Supervisor'),  
        ('admin', 'Administrator'),  
    )  
  
    role = models.CharField(  
        max_length=20,  
        choices=ROLE_CHOICES,  
    )  
  
    department = models.CharField(  
        max_length=150,  
        blank=True,  
        null=True,  
    )  
  
    staff_number = models.CharField(  
        max_length=50,  
        unique=True,  
        blank=True,  
        null=True,  
    )  
  
    student_number = models.CharField(  
        max_length=50,  
        unique=True,  
        blank=True,  
        null=True,  
    )  
    ,
```

Group Exercise 5

- InternshipPlacement
 - student (ForeignKey → CustomUser, role must be Student)
 - company_name
 - start_date
 - end_date

```
class InternshipPlacement(models.Model):

    student = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
        related_name="placements"
    )

    company_name = models.CharField(
        max_length=100
    )

    start_date = models.DateField()
    end_date = models.DateField()

    workplace_supervisor = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
        related_name="workplace_supervisions"
    )

    academic_supervisor = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
        related_name="academic_supervisions"
    )

    is_active = models.BooleanField(
        default=True
    )

    ...
```

What Must Be Achieved By End of Week 2

- Requirements Document:
 - All user stories
 - Functional requirements
 - Non-functional requirements
 - Workflow states
 - Identified entities



Week 2

- GitHub repository:
 - Models implemented
 - At least 3 commits by different members
- Django server running with:
 - CustomUser configured
 - WeeklyLog model created
 - InternshipPlacement created

What You need to know before next Lecture

Technically:

- CustomUser model working
- WeeklyLog model working
- Placement model working
- Requirements documented

Conceptually:

- Clear workflow understanding
- Clear state transitions
- Clear evaluation logic
- Clear system boundaries